

# Compte rendu correction TP2 – Compteur

Clément Chombart

Note : 17/20

Rapport clair, bien illustré, avec un vrai effort d'explication à chaque question plutôt que de simples captures. Ton architecture est correcte : resetn asynchrone directement sur les bascules, restart à **l'intérieur** de la branche rising\_edge donc synchrone, aucune logique combinatoire sur l'horloge. Les deux « bonnes pratiques » que tu notes en conclusion sont exactement les bonnes.

Ta remarque sur tb\_restart non initialisé (qui reste à 'U' et fige la simulation) montre que tu as compris la sémantique à 9 états de std\_logic. C'est un réflexe utile.

Il n'y a que quelques points à reprendre :

1. La dimension de la conversion to\_signed

Tu écris : `V_cible <= std_logic_vector( to_signed(delay,28))- "1" ;`

On en a discuté après le TP donc je serais brève, juste pour le rappel. Le compteur n'a pas nécessité d'être signé, il reste constamment positif. Pour éviter un bit supplémentaire inutile tu peux utiliser la conversion vers un unsigned.

2. Le comptage est décalé d'un cycle

Tu as fait le bon raisonnement en question 7 : « pour compter 200 coups, il faut aller de 0 à 199 ». Tu l'as même traduit dans le code par `V_cible = delay - 1`. Cela implique que `s_end_counter=1` quand `V_data=199` non pas après. Avec ta ligne : `S_end_counter <= '1' when V_data > V_cible else '0';`

`S_end_counter=1` quand `V_data=V_cible+1` (soit `199+1=200`)

Tu comptes alors 1 coup d'horloge en trop. Pour avoir le bon nombre tu as plusieurs options :

`S_end_counter <= '1' when V_data >= V_cible else '0';`

`S_end_counter <= '0' when V_data < V_cible else '1';`

De mon côté j'utilise plutôt la deuxième forme, mais c'est purement arbitraire, la première forme est tout aussi correcte.

3. Question 16 : tu confonds le slack et le délai du chemin

Tu écris « Chemin critique (ns) : 26.243 », puis « Worst negative slack (ns) : 26.243 », puis tu te demandes comment un chemin critique peut dépasser la période d'horloge.

Ce sont deux grandeurs différentes. Le slack est une marge :

$\text{slack} = \text{période disponible} - \text{délai du chemin}$

Un WNS de +26,243 ns signifie qu'il te reste 26 ns de marge, donc que le chemin est très court, pas long (plus le délai est petit plus le slack est élevé). Un WNS positif est toujours bon ; c'est un WNS négatif qui signale une violation. Le délai du chemin lui-même se lit dans la colonne *Total Delay* du rapport, pas dans la colonne *Slack*.

Ensuite, et c'est le point important, regarde la source et la destination de ce chemin :

Source : dbg\_hub/inst/BSCANID.u\_xsdbm\_id/SWITCH\_N\_EXT\_B...

Destination: dbg\_hub/inst/BSCANID.u\_xsdbm\_id/SWITCH\_N\_EXT\_B...

Ce chemin n'appartient pas à ton compteur, cela se voit par « **dbg\_hub** ». Il est interne au debug hub de l'ILA que tu as instancié à la question 15, et il est cadencé par l'horloge du scan JTAG (quelques MHz, donc une grande période et un slack confortable). C'est pour ça que tu obtiens 26 ns : ce n'est pas ton sys\_clk\_pin à 8 ns.

Quand un design contient un ILA, le rapport de timing couvre deux domaines d'horloge : celui de ton compteur (sys\_clk\_pin) et celui du dbg\_hub (JTAG). Le « pire chemin » global tombe fréquemment sur le second, qui ne t'apprend rien sur ton design.

Pour trouver le vrai chemin critique de ton compteur, filtre le rapport sur sys\_clk\_pin (dans Vivado : *Report Timing Summary*, puis regarde la section *Intra-Clock Paths* de sys\_clk\_pin, ou report\_timing -to [get\_clocks sys\_clk\_pin]). Tu tomberas alors sur la chaîne de propagation de la retenue des CARRY4 de V\_data, exactement les composants que tu as bien repérés à la question 13. C'est ce chemin-là qu'il faut commenter.

#### 4. Manque de test/démonstration

Dans les rendus attendus il y a des mesures oscilloscopes et une démonstration. Tu aurais pu utiliser l'oscilloscope pour mesurer la fréquence de clignotement de la LED et faire une courte vidéo du système en fonctionnement. Cela aurait complété les inspections et analyses que tu a réalisé tous au long du TP.